

Specification Document

The following contains in depth information on specific features

Data Structures

WaterBody Class

Purpose

The WaterBody class represents a water body (river, stream, lake) in the watershed system, maintaining its properties and relationships with other water bodies.

Assumptions

- Each water body has a unique name
- Water bodies can be connected in a binary tree structure
- A water body can have dams associated with it

Implementation Details

Inputs:

- Name (string)
- Type (string: "river" or "stream")
- Length (double, in kilometers)
- Basin size (double, in square kilometers)
- Average discharge (double, in cubic meters per second)
- Dam information (vector of Dam objects)

State Changes:

- Pointers to left and right tributaries can be modified
- Parent pointer can be updated
- Dam information can be added or modified

Cases & Expected Behavior:

- New water body creation: Initialize with empty/default values
- Copy construction: Deep copy of all properties except pointers
- JSON serialization: Converts to/from JSON format preserving structure
- Display: Shows all relevant information including connections

Dam Struct

Purpose

The Dam struct stores information about dams associated with water bodies.

Assumptions

- Each dam has a unique name within its water body
- Capacity is measured in MW

Implementation Details

Inputs:

- Name (string)
- Year built (integer)
- Capacity (double, in MW)

Outputs:

- JSON representation of dam data
- Display information through water body

State Changes:

- Properties can be modified through JSON deserialization
- No dynamic memory management required

Navigation & Operations

Tree Navigation

Purpose

Enables users to traverse the watershed structure and inspect water bodies.

Assumptions

- Tree is properly initialized
- All parent/child relationships are correctly set

Implementation Details

Inputs:

- User commands (1-4 for navigation choices)

Outputs:

- Current water body information
- Navigation options
- Success/failure messages

Cases & Expected Behavior:

- Left/Right Navigation: Moves to tributary if available
- Parent Navigation: Moves up the tree if not at root
- Exit: Returns to main menu
- Invalid Input: Displays error message and re-prompts

Adding Water Bodies

Purpose

Facilitates adding new water bodies to the watershed structure.

Assumptions

- Valid input will be provided for required fields
- Proper memory allocation is possible

Implementation Details

Inputs:

- Water body details (name, type, measurements)
- Position choice (left/right tributary)

State Changes:

- Tree structure is modified
- New memory is allocated
- Pointers are updated

Cases & Expected Behavior:

- Empty Tree: Creates root node
- Adding Tributary: Creates new node and updates connections
- Invalid Input: Re-prompts user
- Memory Failure: Displays error message

File Operations

Purpose

Handles construction of watershed data to and from JSON files.

Assumptions

- File system is accessible
- JSON format is valid

Implementation Details

Inputs:

- Filename for save/load operations
- JSON-formatted data (for loading)

Outputs:

- Success/failure status
- JSON file (when saving)

State Changes:

- Tree structure may be completely rebuilt on load
- Existing data is cleared before loading

Cases & Expected Behavior:

- Successful Load: Tree is populated with file data
- Failed Load: Error message, tree remains unchanged
- Successful Save: Data written to file
- Failed Save: Error message, file unchanged

Implementations

1. Dam Class Methods

1.1 Dam::to_json() const

- Description: Serializes the Dam object into a JSON format.
- Parameters: None.
- Returns: A json object containing the dam's name, year built, and capacity.
- Exception Handling: None.

1.2 Dam::from_json(const json& j)

- Description: Deserializes a JSON object into the Dam object.
- Parameters:
 - j: A json object containing the dam details.
- Returns: Void.
- Exception Handling: May throw if JSON does not match expected schema.

2. WaterBody Class Methods

2.1 Constructors and Destructor

- WaterBody()
 - Initializes a new WaterBody with default values.
- WaterBody(const WaterBody& other)
 - Copy constructor, initializes a new WaterBody from another instance.

- `~WaterBody()`
 - Destructor, cleans up dynamic allocations within `WaterBody`.

2.2 `WaterBody::to_json() const`

- Description: Serializes the `WaterBody` object along with its associated `Dam` objects into JSON format.
- Parameters: None.
- Returns: A json object representing the water body and its properties.
- Exception Handling: None.

2.3 `WaterBody::from_json(const json& j)`

- Description: Deserializes a JSON object into the `WaterBody` object, including nested structures like dams and child water bodies.
- Parameters:
 - `j`: A json object containing the water body details.
- Returns: `Void`.
- Exception Handling: May throw if JSON does not match expected schema or if recursive deserialization fails.

2.4 `WaterBody::setParentPointers(WaterBody* parentNode)`

- Description: Sets the parent pointers recursively for a hierarchical water body structure.
- Parameters:
 - `parentNode`: The parent `WaterBody` object.
- Returns: `Void`.
- Exception Handling: None.

3. WatershedTree Class Methods

3.1 `WatershedTree()` and `~WatershedTree()`

- Constructor and Destructor:
 - Initializes a new `WatershedTree` with an empty root.
 - Destructor cleans up all associated `WaterBody` objects.

3.2 `WatershedTree::deleteTree(WaterBody* node)`

- Description: Recursively deletes all nodes in the tree, starting from the given node.
- Parameters:
 - `node`: The root node of the subtree to delete.
- Returns: `Void`.
- Exception Handling: None.

3.3 WatershedTree::loadFromFile(const std::string& filename)

- Description: Loads a watershed structure from a JSON file.
- Parameters:
 - filename: The path to the JSON file.
- Returns: true if loading is successful, false otherwise.
- Exception Handling: Catches and handles file and JSON parsing exceptions.

3.4 WatershedTree::saveToFile(const std::string& filename) const

- Description: Saves the current watershed structure into a JSON file.
- Parameters:
 - filename: The path to the JSON file.
- Returns: true if saving is successful, false otherwise.
- Exception Handling: Handles file opening exceptions.

3.5 WatershedTree::addWaterBody()

- Description: Interactively adds a new WaterBody to the watershed, with user input defining its properties and position within the tree.
- Parameters: None.
- Returns: Void.
- Exception Handling: Handles input exceptions and ensures consistent tree structure.